

Rapport individuel — Itération 2

Alexis Briend (abd7852) — Projet Long SHDL

11 mai 2026

Contexte

Responsable de la **structure générale** et de l'**interprétation du SHDL** (SCRUM-23). Objectif sprint 2 : remplacer le parser à descente récursive du sprint 1 par une version *LL(1) table-driven* stricte, et brancher concrètement parser et simulateur via une couche de conversion. Atteint sur 82 commits.

Activités réalisées

- **Parser LL(1) table-driven** (`parser/ll1/taledriven/`) : table d'analyse $M[NT, T]$ bâtie depuis FIRST/FOLLOW (`TableBuilder`, lookup $O(1)$), driver à pile générique sans *switch* (`CstParser`). Grammaire gelée par `GrammarFreezeTest`, sans conflit FIRST/FIRST ni FIRST/FOLLOW.
- **Concrete Syntax Tree** en remplacement de l'AST : types `sealed CstNode = CstLeaf | CstInternal` (records immuables), navigation `first/allOf/has`, offsets préservés, trivia filtrés.
- **Conversion CST → simulateur.Module** (`parser/conversion/`, spec docs/specs/2026-05-06-cst-conversion) combinatoire scalaire stricte. Hors-scope (vecteurs, mémoire, appel de module, doublons) rejetés par `ConversionException` typée avec offset et type de nœud.
- **Bug fix amont** : `simulateur.Erwan.OR()` retournait `Op.AND`, corrigé avec test sentinelle anti-régression.

Résultats

- **193 tests JUnit verts** (+86 vs sprint 1), 39 classes : parser (offsets, ε -productions, erreurs avec position), conversion (cas nominaux `ET/OU/NOT`, rejets exhaustifs), bout-en-bout (`xor` via `FileSimulateur`).
- **14 fichiers Java** ajoutés, intégration réussie avec Erwan (`parser/lexer/`) et Mati (`simulateur.Module`, `FileSimulateur`).

Difficultés et réussites

Difficulté principale : *instabilités du code amont* (stubs successifs côté Erwan, bug `OR` non détecté sans un test e2e sur table de vérité `XOR`). La discipline TDD a été payée plusieurs fois ce sprint. Réussite principale : **séparation stricte** parser / CST / conversion / IR. Le CST sert d'interface contractuelle : la grammaire peut évoluer sans casser la conversion, la conversion peut grandir vers le séquentiel sans toucher au parser.

Manuel utilisateur

Pré-requis : JDK 25, JUnit 4.13.2, Hamcrest 1.3. Pipeline d'utilisation :

```
CstNode cst      = CstParser.parse(shdlSource);      // ParsingException
Module module    = Conversion.convert(cst);          // ConversionException
FileSimulateur sim = new FileSimulateur(module.Plan);
```

Les exceptions exposent l'`offset()` et un code structuré, exploitables par l'IHM pour le surlignage. Code sur la branche `feature/parser-ll1-table-driven`.